DeployGuard
Deployment & Orchestration Architecture Review

Current-state audit, complexity analysis, industry-backed simplification plan, migration effort, and reliability estimate

Current DeployGuard branch: test1
Audited commit: e9ce648f227cea12d56a8207d50e8de71ce96613
Commit: Reconcile managed MySQL host grants
Report date: 4 September 2026

Important: The success-rate figures in this report are engineering planning estimates, not historical production SLOs. The available executable evidence is insufficient to calculate a statistically valid observed success rate.

1. Executive Summary

Conclusion: DeployGuard has strong functionality and several mature safety mechanisms, but the cloud deployment path is over-coupled. Terraform currently acts as infrastructure provisioner, application release mechanism, database-readiness controller, managed-MySQL grant reconciler, lifecycle engine, and part of the diagnostic system. That is the main source of complexity.

Metric	Current	Proposed target	Assessment
Deployment/orchestration complexity	88/100	48-55/100	About 38-45% reduction
Overall architecture score	62/100	84-88/100 after certification	Functionally strong, structurally over-coupled today
Expected platform-orchestration success*	70-80%	90-95%	Planning estimate under valid app + healthy cloud
Migration size	N/A	25-35% of deployment engine	Do not rewrite detection/Railpack
Engineering effort	N/A	60-95 focused hours	Implementation + AWS lifecycle certification

Recommended architectural boundary: Railpack = build. GitHub Actions = execution. Terraform = infrastructure topology and destroy. ECS service revisions/task definitions = releases. DeployGuard = orchestration, policy, evidence, lifecycle state. Managed DB reconciliation = a dedicated runtime prerequisite.

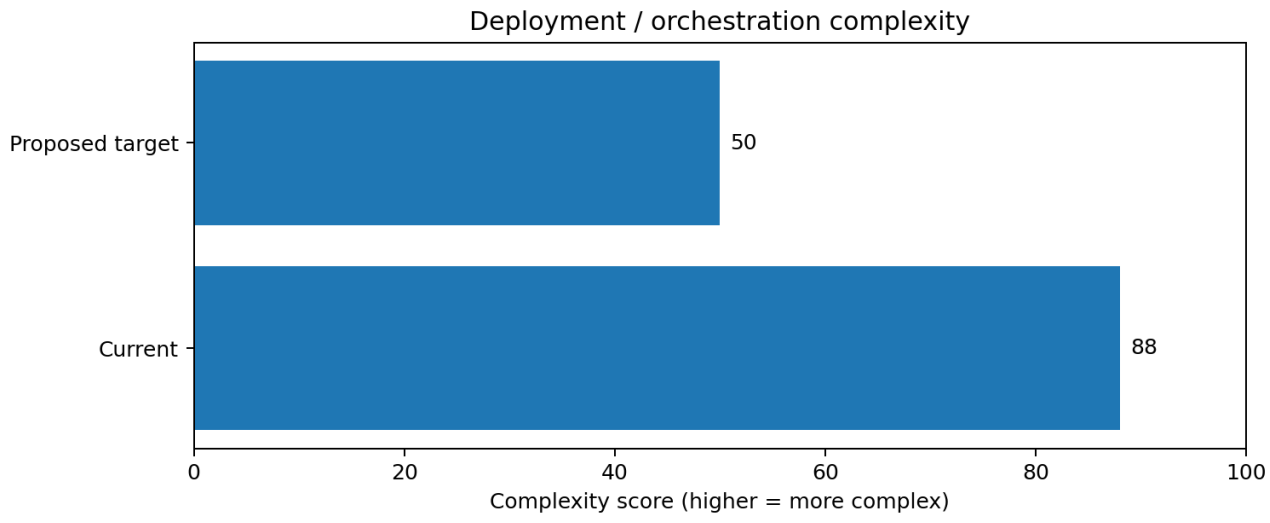


Figure 1. Current versus proposed deployment/orchestration complexity score.

2. Scope, Method, and Evidence

- Audited DeployGuard test1 at commit e9ce648f227cea12d56a8207d50e8de71ce96613.
- Inspected the current reusable GitHub Actions workflow and the Terraform runtime implementation.
- Inspected current backend lifecycle control for deploy, retry, rollback, reset-fresh, and destroy.
- Included executable evidence already available from the 20 full-stack combination-app deployment run, including the MySQL ECS host-grant failure and its repair.
- Cross-checked the redesign against official HashiCorp Terraform guidance, AWS ECS deployment primitives, AWS Copilot behavior, AWS GitHub Actions, and Google Cloud Deploy release/rollout concepts.

Evidence limitation: Redeploy, rollback, and destroy are not yet fully certified by current-SHA end-to-end AWS execution. Therefore the report distinguishes confirmed source behavior from end-to-end verification.

3. Current DeployGuard Cloud Architecture

3.1 Normal deploy path

```
Repository / configured services
  -> DeployGuard control-plane admission
  -> exact source SHA resolution
  -> GitHub Actions reusable workflow
  -> pinned Railpack build
  -> local application runtime validation
  -> ECR immutable images
  -> Terraform materialization
  -> terraform init + validate + plan + apply
  -> ECS / ALB / target groups / CloudWatch / security groups
  -> managed DB + EFS + Cloud Map + Secrets Manager
  -> DB readiness / MySQL grant reconciliation
  -> ECS runtime + public verification
  -> release evidence artifact
  -> DeployGuard control-plane finalization
```

Code evidence: .github/workflows/deployguard-reusable.yml and infrastructure/railpack-runtime/main.tf at e9ce648...

3.2 Redeploy / retry behavior

A normal deploy call uses the same dispatch action "deploy". Retry replays the lifecycle type of the last failed operation: destroy, rollback, or deploy. This is valuable for consistency, but it also means ordinary code redeploys continue to traverse the same large cloud/Terraform path rather than a smaller ECS release path.

3.3 Rollback behavior

Rollback is structurally strong in one important way: DeployGuard persists immutable image digests and runtime-configuration revision identities and rejects rollback targets that do not contain a complete immutable service set. However, rollback still enters the reusable workflow and the Terraform plan/apply path, then performs runtime verification. Therefore rollback remains coupled to the full Terraform graph.

3.4 Destroy behavior

Destroy is intentionally Terraform-owned. The workflow runs terraform destroy, verifies project/generation cleanup evidence, handles runtime secrets created outside Terraform, and then allows DeployGuard control-plane cleanup. Keeping destroy primarily on Terraform is reasonable and is retained in the proposed design.

3.5 Managed database behavior

Managed PostgreSQL/MySQL/MongoDB is modeled as persistent infrastructure. For MySQL, the current repair adds an ECS-side grant reconciler which waits for database health, creates or updates deployguard@'%', refreshes the password, grants access to the application database, verifies login, and blocks application creation if reconciliation fails. This solves the specific ECS task-IP rejection but increases Terraform's procedural responsibility.

4. What the Current Design Gets Right

- Exact source identity: deploy input validates a 40-character source SHA and checks out that exact revision.
- Pinned build tooling: Railpack is pinned, checksum-verified, and used as the build authority.
- Immutable artifacts: ECR images and rollback targets use immutable digests rather than mutable latest-style identity.
- Local runtime validation before AWS: application containers are exercised before Terraform/ECS deployment.
- Service-scoped runtime ownership: per-service environment, secrets, ports, and database attachment are carried through the runtime contract.
- Runtime verification and evidence: the workflow preserves service-specific failure evidence and release-result artifacts.
- Managed-MySQL repair: the current commit explicitly reconciles host grants for changing ECS task IPs.
- Rollback safety checks: old releases without immutable runtime configuration are rejected rather than guessed.
- Destroy has an explicit authoritative-generation contract rather than deleting arbitrary resources.

5. Current Complexity, Defects, and Uncertainty

Issue	Status	Current evidence	Impact
Terraform owns too many layers	High	Terraform provisions infrastructure and also performs runtime readiness, DB grant reconciliation, release convergence, and failure signaling.	Primary complexity driver
Volatile releases share lifecycle with stable/stateful infra	High	An application image change can traverse the graph containing ALB, security, DB/EFS, service discovery, and other stable resources.	Large blast radius / longer reasoning path

Issue	Status	Current evidence	Impact
Global DB readiness barrier	Confirmed defect	All application ECS services depend on terraform_data.database_readiness, so a DB prerequisite failure can block an unrelated service before creation.	Breaks isolation for pre-creation DB failures
Reset & Deploy Fresh is ordinary deploy	Confirmed defect	resetAndDeployFresh currently dispatches "deploy" without an actual reset/reconciliation lifecycle.	Fresh semantics are not guaranteed
Managed DB admission reconciliation gap	Confirmed source concern	Admission loads a managed database by provider; it does not visibly enforce an authoritative reconciliation state before deployment.	Stale DB metadata can re-enter the runtime contract
Terraform plan error labeled apply failure	Confirmed defect	A plan failure currently emits DG_TERRAFORM_APPLY_FAILED / terraform_apply.	Diagnostic ambiguity
Release-state pin mismatch in validation repo	Release blocker	The 20-app caller workflow observed during this audit still references reusable workflow 05afec5..., not current e9ce648....	AWS retest can accidentally execute old code
Redeploy / rollback / destroy certification	Unverified	Source paths exist, but current-SHA end-to-end AWS proof is incomplete.	Certification uncertainty

5.1 Why terraform init / plan / apply became difficult

- The commands stayed simple; the dependency graph did not. Each new runtime requirement was added behind or inside Terraform.
- Persistent databases introduced state that Terraform cannot fully see: users, grants, schema, migrations, and retained data inside EFS.
- Runtime health became a Terraform prerequisite through terraform_data + local-exec polling.
- Lifecycle semantics were layered onto the same graph: deploy, rollback, destroy, retry, and reset-fresh all need different state behavior.
- Failure ownership became part of the IaC path, so shell/Terraform/AWS failures must map into stable DeployGuard failure codes.
- The result is a declarative infrastructure graph carrying procedural application-controller responsibilities.

6. Current-State Scoring

Dimension	Score	Reason
Functional breadth	86/100	Deploy, retry, rollback, destroy, multi-service runtime, managed DB, verification, evidence, cost plan, immutable releases.
Deployment/orchestration complexity	88/100	Too many responsibilities cross Railpack -> GitHub Actions -> Terraform -> ECS/DB -> verification.
Separation of responsibilities	46/100	Build is well-separated, but Terraform and runtime/lifecycle ownership are still mixed.
Maintainability	52/100	Strong contracts/tests help, but changes to one lifecycle edge can affect a large graph.
Failure isolation	68/100	Runtime service failure isolation has executable evidence; DB-precondition isolation still has a structural defect.
Certification confidence	60/100	Deploy evidence exists; remaining lifecycle paths and current MySQL repair need current-SHA AWS proof.
Overall architecture	62/100	Good FYP capability and serious engineering work, but current cloud lifecycle is over-coupled.

7. Industry Evidence for Simplification

7.1 HashiCorp: keep Terraform scope and blast radius small

HashiCorp recommends separating infrastructure by volatility and separating stateful resources such as databases from stateless compute. It also warns that Terraform is primarily designed for immutable infrastructure and recommends purpose-built solutions for post-apply operations before using provisioners. This maps directly to DeployGuard's current pain: database readiness and grant reconciliation are procedural runtime tasks embedded in Terraform. [R1][R2]

7.2 AWS ECS: a release is naturally a task-definition/service update

AWS provides a direct release primitive: register a task definition, update an ECS service, wait for service stability, and optionally use the deployment circuit breaker with rollback. The official AWS GitHub Action performs the same basic operation: register the task definition and deploy it to an ECS service. [R3][R4] [R5]

7.3 AWS Copilot: IaC may remain, but service deploy stays service-scoped

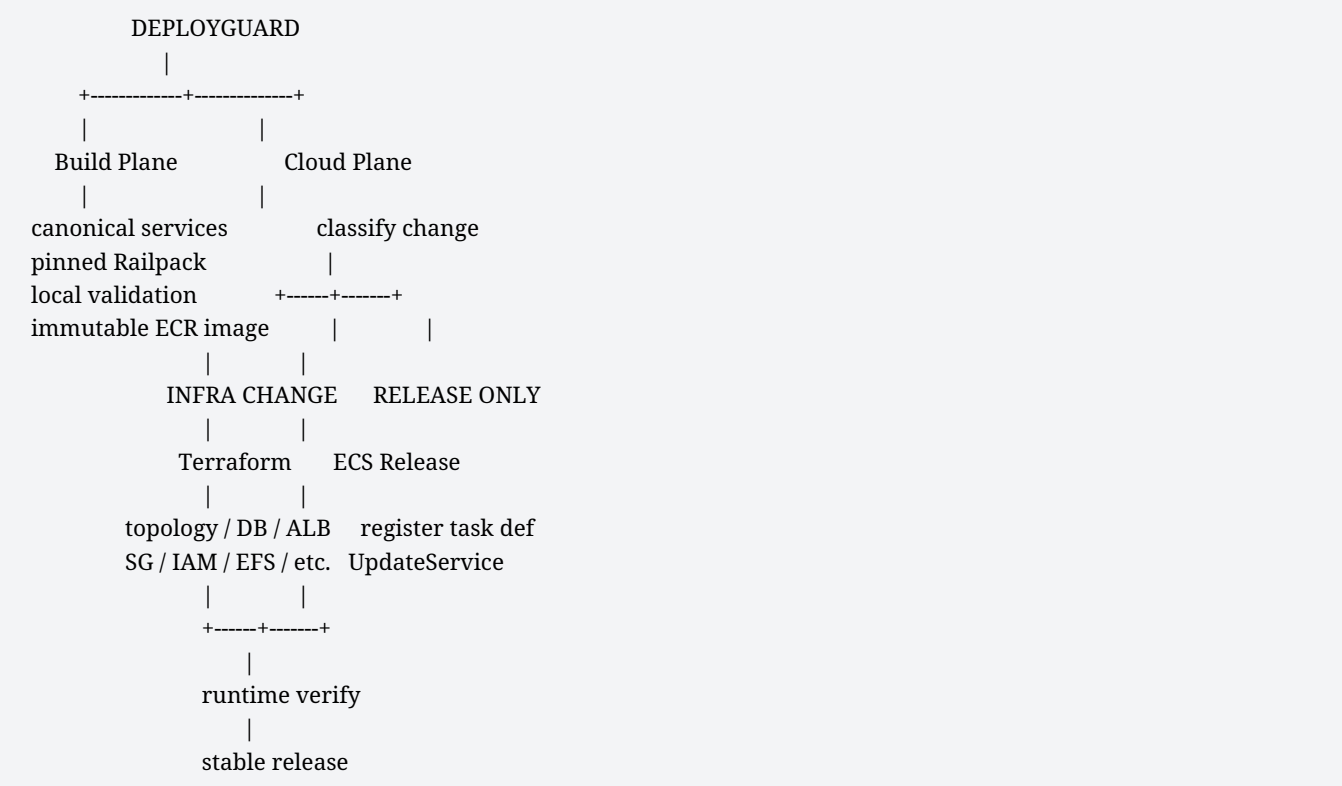
AWS Copilot is an important counterexample to an overly absolute rule. Copilot still packages CloudFormation during service deploy, but the deploy remains centered on the service: build image, push to ECR, package manifest/addons, and create/update the ECS task definition and service. Therefore the recommendation is not "never use IaC during deploy"; it is "do not make the full persistent infrastructure graph the release controller." [R6]

7.4 Google Cloud Deploy: separate releases and rollouts from target infrastructure

Google Cloud Deploy models an application release separately and creates rollouts that associate that release with a deployment target. This is the same architectural separation proposed for DeployGuard: infrastructure/target existence is one concern; release promotion and verification are another. [R7]

8. Proposed DeployGuard Architecture

Design rule: Do not redesign the part that finally works. Freeze repository detection/runtime configuration and Railpack build behavior. Simplify only the cloud release/orchestration boundary.



8.1 Ownership model

Component	Proposed owner	Responsibility
Railpack	Build plane	Build/start intelligence and OCI image creation.
GitHub Actions	Execution plane	Runs deterministic

Component	Proposed owner	Responsibility
		release/infrastructure jobs with OIDC credentials and evidence capture.
Terraform	Infrastructure controller	Create/update topology and persistent/shared resources; perform final destroy.
ECS task definitions + UpdateService	Release controller primitive	Normal deploy/redeploy/rollback of immutable application revisions.
DeployGuard backend	Orchestrator	Admission, change classification, lifecycle state, release history, retries, ownership, evidence.
Managed DB controller	Runtime prerequisite	Wait for DB, reconcile credentials/grants, verify app credentials, mark DB ready.

8.2 Lifecycle flows after simplification

Lifecycle	Proposed path	Terraform?
First deploy	Build -> ECR -> ensure infra -> DB reconcile if attached -> ECS release -> verify -> stable	Yes, if infra missing
Normal redeploy	Build -> ECR -> register task definition -> UpdateService -> wait -> verify	No
Rollback	Select stored immutable task definition/image/runtime revision -> UpdateService -> wait -> verify	No
Retry	Resume/re-run the failed domain using the same immutable inputs; do not rebuild unless failure was build/source related	Only if infra failed
Reset & Deploy Fresh	Explicit reset policy -> clean/reconcile runtime resources -> preserve/delete DB according to contract -> ensure infra -> release	As required
Destroy	Terraform destroy -> verify resources gone -> delete non-Terraform secrets/artifacts -> control-plane finalization	Yes

8.3 Change classifier - the key simplifier

DeployGuard should decide whether the new canonical deployment contract changes infrastructure topology or only the application release. A small immutable topology fingerprint can include service IDs, ports that affect target groups/security rules, database attachment/engine, public entrypoint, and other Terraform-owned fields. Image digest, normal runtime ENV/secrets, and source SHA are release-owned unless they require an infrastructure field change.

```
if topology_fingerprint_changed:
   run Terraform ensure/update
else:
   skip Terraform

always for an application release:
   register immutable task definition
   ECS UpdateService
   wait for deployment
   verify runtime/public endpoint
   persist stable release
```

9. Expected Simplification

Area	Current	Proposed	Reduction
Normal redeploy	Full Terraform materialize/init/validate/plan/apply + runtime verification	ECS task-definition update + service deployment + verification	Very large
Rollback	Immutable target but still traverses Terraform plan/apply	Update ECS service to prior immutable release + verify	Very large
Database readiness	Terraform local-exec/readiness barrier plus ECS sidecar reconciliation	Dedicated DB prerequisite/controller; only attached service waits	Large
Failure isolation	Terraform dependency graph can couple unrelated services	Release controller handles each service independently; infra changes are explicit	Large
Destroy	Terraform destroy + cleanup	Mostly unchanged	Small
Detection/build	Railpack + local validation	Unchanged	Zero migration risk

Expected complexity reduction: from approximately 88/100 to 48-55/100. The largest gain is not fewer AWS resources; it is fewer lifecycle responsibilities crossing the same Terraform graph.

10. Success-Rate Estimate: Before vs After

Definition: These percentages estimate the probability that DeployGuard itself completes orchestration successfully when the repository is valid, the built application is healthy, credentials are correct, and AWS/GitHub/Railpack are available. They do not count genuine application defects or provider outages as platform failures.

Lifecycle	Current estimate	Proposed estimate	Reason
Deploy	72-82%	90-95%	Initial deploy still has infra complexity, but runtime release logic becomes smaller and DB gating becomes explicit.
Redeploy	68-78%	92-96%	Removes repeated Terraform graph traversal for ordinary image/runtime revisions.
Rollback	65-75%	92-96%	Becomes a direct immutable release switch rather than a full IaC convergence operation.
Destroy	78-88%	82-90%	Still Terraform-heavy, so improvement is intentionally modest.
Overall lifecycle	70-80%	90-95%	Fewer coupled failure edges and clearer ownership.

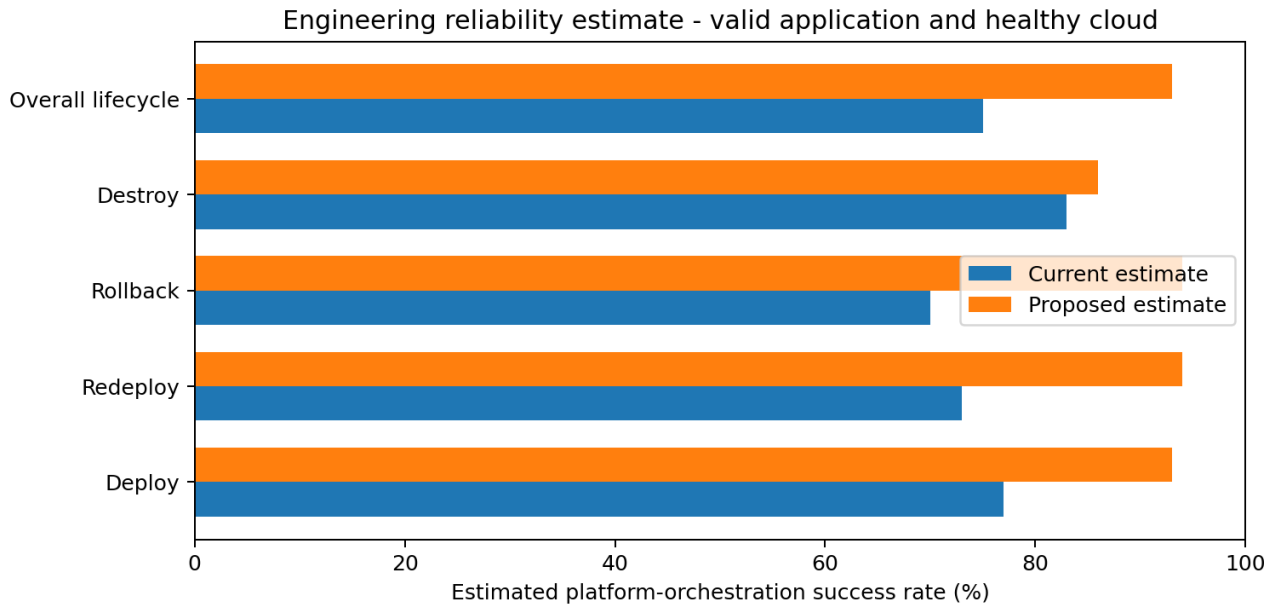


Figure 2. Midpoint visualization of the engineering reliability ranges above.

Why this is an estimate, not a measurement: the current evidence set does not contain enough comparable current-SHA deployments across deploy/redeploy/rollback/destroy to calculate a statistically meaningful observed rate. A real SLO should be computed later from a certification matrix and production telemetry.

11. Migration Plan - Incremental, Not a Rewrite

Safety principle: Keep the current working engine available during migration. Replace one lifecycle path at a time, certify it, then remove the old path. Do not modify Railpack/build detection while changing cloud orchestration.

Phase	Work	Focused effort
0. Freeze contracts + baseline tests	Lock current runtime contract, topology fields, release evidence schema, and known-good deploy fixtures.	4-6 h
1. ECS release controller	Generate/register task definitions from canonical service revisions; UpdateService; wait; verify; persist evidence.	12-18 h
2. Move normal redeploy	Route release-only changes around Terraform. Add topology-fingerprint decision and idempotent retry.	8-12 h
3. Move rollback	Map immutable historical revision -> ECS task definition/service update. Preserve release pointer only after verification.	8-12 h
4. DB prerequisite controller	Move DB wait/grant reconciliation out of Terraform procedural orchestration; block only attached service.	10-15 h
5. Fix reset-fresh semantics	Implement actual reset/reconciliation policy with explicit persistent-DB behavior.	6-10 h
6. Terraform scope cleanup	Remove release-only triggers/provisioners, keep topology/persistent resource ownership and destroy.	6-10 h
7. AWS certification	Run deploy, task-IP replacement, redeploy, rollback, DB persistence, reset, failure isolation, destroy.	12-20 h

Total planning estimate: approximately 60-95 focused engineering hours, or roughly 8-12 full working days at 7-8 focused hours/day. With effective Codex assistance, implementation may be faster, but AWS certification should not be compressed because the goal is architecture confidence, not just code completion.

11.1 Expected code-change size

- Approximately 25-35% of the deployment/orchestration engine changes.

- The Railpack/build subsystem should remain effectively untouched.
- Most changes concentrate in the reusable workflow, Terraform ownership boundary, backend lifecycle orchestration, and new ECS/DB controller helpers.
- Destroy can remain close to its current architecture, reducing migration risk.

12. Acceptance Criteria Before Calling the New Architecture Better

1. First deploy creates infrastructure and reaches the public endpoint.
2. A code-only redeploy performs no Terraform plan/apply and changes only the intended ECS release.
3. A topology change correctly triggers Terraform and then deploys the new release.
4. Replacing an ECS task changes its private IP and a MySQL-attached service still connects successfully.
5. An unrelated service deploys/verifies even when a managed-DB prerequisite fails for the attached service, if the product contract permits partial service success.
6. Rollback uses the exact prior immutable image/runtime revision and performs no source rebuild.
7. A failed release never moves the stable-release pointer.
8. Reset & Deploy Fresh has a real reset/reconciliation contract and does not silently behave like ordinary deploy.
9. Destroy is idempotent after partial destroy and cleans control-plane state only after verified AWS deletion.
10. Every failure edge records lifecycle, stage, service, stable code, owner, and evidence without mislabeling Terraform plan as apply.

13. Risks and Controls in the Proposed Design

Risk	Control
Terraform and ECS both try to own task_definition	Explicit ownership: Terraform ignores the release-owned task_definition field or the service resource is structured so release updates are expected and do not create drift fights.
Topology classifier misses an infra-affecting change	Versioned topology fingerprint + contract tests + fail closed: uncertain changes run Terraform rather than skipping it.
Rollback task definition references stale secret versions	Persist exact runtime-config revision and validate managed-DB secret/version compatibility before promotion.
DB controller becomes another hidden state machine	Keep it very small: wait -> reconcile -> verify -> READY/FAILED; persist deterministic evidence; no repository detection.
Migration destabilizes working deploy path	Feature-gated incremental migration; certify each lifecycle before deleting old behavior.

14. Final Recommendation

Proceed with the simplification, but do not perform a wholesale rewrite. The evidence supports a targeted ownership correction rather than another complete architecture change. Keep the build plane

that now works. Keep GitHub Actions. Keep Terraform. Keep ECS. Change only which subsystem owns which lifecycle responsibility.

> **Architecture to freeze:** Railpack = build; GitHub Actions = execution; Terraform = infrastructure topology + destroy; ECS = application release revisions; DeployGuard = orchestration/evidence; DB controller = managed-database runtime readiness.

This approach is simpler because it follows native platform boundaries instead of inventing more detection or abstraction. It also preserves DeployGuard's FYP value: the project still owns deployment policy, lifecycle history, environment/secrets, verification, rollback selection, failure ownership, monitoring, admin functions, SNS notifications, infrastructure views, and pricing; it simply stops using Terraform as the runtime release controller for every application change.

15. Sources and Code Evidence

15.1 DeployGuard source evidence

- Deploy-Guard-dev / test1 @ e9ce648f227cea12d56a8207d50e8de71ce96613 - current audited head.
- .github/workflows/deployguard-reusable.yml - exact source validation, Railpack build, Terraform materialization/init/validate/plan/apply/destroy, runtime verification, result evidence.
- infrastructure/railpack-runtime/main.tf - ECS application/database resources, ALB/TG, EFS, service discovery, DB readiness barrier, MySQL grant reconciler.
- backend/src/projects/railpack-deployment.service.ts - deploy/retry/rollback/reset-fresh/destroy admission and immutable release behavior.
- 20-fullstack-three-tier-combination-apps/.github/workflows/deployguard.yml - observed caller still pinned to reusable workflow 05afec5... during this audit.

15.2 Industry references

[R1] HashiCorp - Workspace Best Practices - https://developer.hashicorp.com/terraform/enterprise/workspaces/best-practices

[R2] HashiCorp - Perform post-apply operations using provisioners - https://developer.hashicorp.com/terraform/language/provisioners

[R3] AWS ECS - RegisterTaskDefinition API - https://docs.aws.amazon.com/AmazonECS/latest/APIReference/API_RegisterTaskDefinition.html

[R4] AWS ECS - Update service parameters / deployment circuit breaker - https://docs.aws.amazon.com/AmazonECS/latest/developerguide/update-service-parameters.html

[R5] AWS GitHub Action - amazon-ecs-deploy-task-definition - https://github.com/aws-actions/amazon-ecs-deploy-task-definition

[R6] AWS Copilot - svc deploy - https://aws.github.io/copilot-cli/docs/commands/svc-deploy/

[R7] Google Cloud Deploy - Overview of releases and rollouts - https://docs.cloud.google.com/deploy/docs/overview

[R8] HashiCorp - lifecycle ignore_changes - https://developer.hashicorp.com/terraform/language/meta-arguments/lifecycle

Report status: Architecture recommendation suitable for implementation planning. Final success-rate/SLO claims require the post-migration certification matrix and repeated current-SHA AWS runs.